

PROSPECTO: LIBELLULA

(31/08/2026)

LIBELLULA

A experiência geral da especulação financeira tem chegado à conclusão de que, à longo prazo, a gestão do risco é mais imperativa ao sucesso financeiro do operador do que a capacidade preditiva.

Entretanto, com o advento do poderio das tecnologias de Machine Learning, uma parcela majoritária do processamento de IA relativo às finanças quantitativas tem sido devotado à métodos preditivos.

A proposta central do Libellula é desviar o foco do emprego das tecnologias de Machine Learning, migrando-o do “*preditivismo*” para algo mais fundamental: **a gestão de risco**.

Os operadores, geralmente, estão cientes das deficiências dos métodos tradicionais de gestão de banca, especialmente quando referentes ao trading de alta intensidade (que pode comportar dezenas de operações no *intraday*). Estes métodos estáticos são insuficientes para lidar com uma rotina de operações intensa em mercados extremamente líquidos e voláteis.

O projeto Libellula visa corrigir esta falta, promovendo uma arquitetura que trabalha baseada na obtenção de dados probabilísticos subjacentes às séries temporais e na tradução destes dados em uma política adaptativa rápida de gestão de banca.

O objetivo do Libellula não é construir um modelo que simplesmente preveja o mercado. O objetivo é utilizar uma previsão probabilística do mercado e uma caracterização explícita da incerteza para permitir que um agente de RL aprenda a alocar dinamicamente o risco e o capital de cada operação.

Arquitetura geral do projeto

O Libellula é um algoritmo que objetiva:

1. Perceber os riscos subjacentes percebidos em séries temporais de cotações de ativos líquidos.
2. Traduzir estes riscos em uma política de gestão de banca (gestão de risco operacional) orientada a quatro fatores:
 - a. Position size
 - b. Stop loss
 - c. Take profit

De certa forma, é bastante simples.

Terminologia interna do projeto

1. **Dataset original:** é como se chama o dataset inicial de todo o workflow. Ele é apenas a cotação OHLC de um ativo serial qualquer.

2. **Dataset técnico:** é um dataset que contém a cotação OHLC do ativo mais os indicadores técnicos gerados pelo arquivo. Este dataset é o insumo fundamental para o trabalho do modelo TFT preditivo.
3. **Envelope de risco:** é um conjunto de ferramentas de análise estatística de séries temporais. O propósito deste envelope é abstrair os fenômenos estatísticos subjacentes das cotações e organizá-los em um novo dataset. As ferramentas são correntemente:
 - a. **CVaR:** Também conhecido como *Expected Shortfall*, o CVaR mede o risco de cauda de uma carteira. Ele calcula a média das perdas que excedem o limite do VaR (Value at Risk). Enquanto o VaR diz "qual é a perda mínima nos 5% piores cenários", o CVaR responde "qual é a perda média se entrarmos nesses 5% piores cenários".
 - b. **EVT:** A Teoria dos Valores Extremos é um ramo da estatística dedicado a modelar e prever eventos raros de altíssimo impacto (como crises financeiras ou desastres naturais – eventos conhecidos como “cisne negro”). Em vez de focar no comportamento médio dos dados (regidos pelo Teorema Central do Limite), a EVT foca estritamente nas caudas das distribuições para estimar a probabilidade de desvios extremos.
 - c. **EPF:** Trata-se do campo de estudo e aplicação de modelos preditivos para estimar os preços da energia elétrica. É um desafio complexo devido a características únicas desse mercado, como a impossibilidade de armazenamento em larga escala, alta volatilidade, sazonalidade extrema (diária, semanal, anual) e picos abruptos de preço (*spikes*).
 - d. **Quantile forecasting:** Diferente da previsão tradicional que busca estimar a média (um valor único futuro), a Previsão de Quantis estima pontos específicos da distribuição de probabilidade futura (ex: o quantil 90%). Isso permite construir intervalos de confiança assimétricos e entender melhor a incerteza e os riscos associados a cenários de cauda.
 - e. **PDFD:** A Distância entre Funções de Densidade de Probabilidade é uma métrica (como a divergência Kullback-Leibler ou a distância de Wasserstein) usada para quantificar a diferença entre duas distribuições de probabilidade inteiras, e não apenas entre seus pontos estatísticos isolados (como média ou variância).
4. **Dataset do envelope de risco:** são os datasets gerados por cada uma das ferramentas do envelope.
5. **Dataset concatenado do envelope de risco:** como cada uma das ferramentas do envelope gera um dataset, precisamos concatena-los. Este é o dataset concatenado do envelope de risco.
6. **Dataset preditivo:** é o dataset gerado pelo modelo TFT preditivo. Ele contém previsões geradas via ‘rolling_window’.
7. **Dataset básico para modelo RL de gestão de banca:** é a combinação de dois datasets:
 - a. O dataset concatenado do envelope de risco.

b. O dataset preditivo.

Este dataset será usado para o treinamento do modelo RL de gestão de banca.

8. Modelo TFT preditivo: é um modelo estilo Temporal Fusion Transformers para previsão de séries temporais.

9. Modelo RL de gestão de banca: este modelo ainda não existente faz algo complexo. Ele toma o dataset básico para modelo RL de gestão de banca e usa para treinamento. A proposta deste modelo é muito simples:

- a. Ele deve entregar 3 outputs rudimentares para o gerenciamento dinâmico da banca:
 - i. Position size.
 - ii. Stop loss.
 - iii. Take profit.

Workflow do Libellula

O algoritmo se executa através de fases distintas:

1. Obtenção dos dados (*dataset original*).
2. Tratamentos dos dados (método 'setup()') e criação dos indicadores técnicos. Geração de um novo dataset (OHLC + indicadores técnicos): o *dataset técnico*.
3. Este *dataset técnico* será levado para ser processado por dois algoritmos diferentes.
 - a. O primeiro é o algoritmo do *modelo TFT preditivo*, que receberá o *dataset técnico* (OHLC + indicadores técnicos) e gerará um novo dataset de previsões: o *dataset preditivo*.
 - b. O segundo é na verdade vários algoritmos: os algoritmos que compõem o *envelope de risco*. Cada um destes algoritmos que o compõem constitui-se de uma ferramenta estatística/probabilística que analisa o dataset com as cotações + indicadores técnicos e gera, como output, um novo conjunto de datasets, chamados *datasets do envelope de risco*, contendo os números extraídos do processo.
 - i. Ainda nesta mesma pasta (do *envelope de risco*), há um algoritmo que concatena os datasets gerados por cada uma das ferramentas do envelope de risco. Logo, temos um novo dataset: o *dataset concatenado do envelope de risco*.
4. Então, ocorre a concatenação dos resultados do modelo TFT preditivo + resultados do envelope de risco, o que resulta no *dataset básico para modelo RL de gestão de banca*.
5. O modelo RL de gestão de banca treina usando os dados deste último dataset.
6. Geração de resultados.

Estas últimas duas etapas do workflow não estão nem iniciadas em seu processo de programação.

Recapitulando: nesta arquitetura, um dataset contendo dados financeiros rudimentares (cotações em *Open High Low Close*), ou seja, o *dataset original*, é analisado por uma dupla linha de workflow:

- a. A primeira envolve a criação de indicadores técnicos e concatenação do novo dataset (OHLC + indicadores técnicos). Este novo dataset, chamado *dataset técnico*, concatenado deve ser empregado no treinamento do *modelo TFT preditivo*.
- b. A segunda é a do *envelope de risco*: uma sucessão de ferramentas estocásticas e probabilísticas que visa abstrair fenômenos probabilísticos subjacentes do dataset inicial. Estas ferramentas geram datasets que posteriormente também serão concatenados e transformados num só dataset final, chamado *dataset básico para modelo RL de gestão de banca*.

Ao final, os dois datasets, a saber:

- O dataset gerado pelo modelo preditivo.
- O dataset gerado pelo envelope de risco.

... são concatenados num só: o *dataset básico para o modelo RL de gestão de banca*.

Durante o treinamento, o modelo de Reinforcement Learning recebe o estado disponível no instante t , produz uma ação de gestão de risco e capital e observa as consequências dessa decisão em relação à evolução posterior do mercado. Essas consequências são utilizadas pelo ambiente de treinamento para calcular o retorno e o risco associados à ação e, consequentemente, o reward utilizado para o aprendizado da política. Assim, informações futuras participam da avaliação da decisão durante o treinamento, mas não fazem parte do estado disponível ao agente no momento em que a decisão é tomada

Por causa de uma certa displicência de minha parte, eu incluí, no notebook do modelo TFT preditivo, um algoritmo que concatena as previsões do modelo TFT preditivo aos datasets concatenados gerados pelo envelope de risco. Obviamente, este algoritmo não deveria estar lá. Ele será removido para outro lugar em breve.

De forma resumida: as três principais camadas do sistema possuem responsabilidades distintas. O modelo TFT é responsável pela previsão do comportamento futuro do ativo; o envelope de risco é responsável por caracterizar a incerteza e o risco associados à previsão, sem tomar decisões operacionais; e o modelo de Reinforcement Learning é responsável pela decisão de gestão de capital e risco. Dessa forma, nenhuma dessas camadas precisa reproduzir integralmente a função das demais.

O estado fornecido ao modelo de Reinforcement Learning é composto fundamentalmente pelas informações produzidas pelo modelo preditivo e pelas ferramentas do envelope de risco. A intenção é que o RL opere sobre uma representação já processada do estado probabilístico do mercado, em vez de receber diretamente as cotações OHLC e os

indicadores técnicos utilizados pelo TFT. Dessa maneira, o RL concentra seu aprendizado na gestão da exposição ao risco e do capital, evitando que sua função seja confundida com a função preditiva do TFT.

Uma vez que tudo esteja concatenado, o modelo RL de gestão de banca inicia seu treinamento para a entrega dos outputs desejados.

Considerações de estado atual: Código de teste Vs. Código hospedado

Há ainda diferenças fundamentais de código entre o que existe já em forma de notebook e o que deve ser implementado em CI/CD robusto. O que existe agora são códigos de teste de conceito, por assim dizer: códigos que não podem ser implementados em escala industrial de live trading, mas que são bons para testarmos a teoria geral da arquitetura do projeto.

Mas o código que estamos usando atualmente não pode ser implementado. Ele é apenas um código para teste de conceito, e não para implementação. O código atual serve ao propósito demonstrativo de revelar a teoria arquitetônica e matemática do projeto, e não ao propósito de hospedagem e lançamento em forma de serviço.

Assim que toda esta teoria estiver devidamente testada, poderemos fazer alterações estruturais no código para orientá-lo a produção, definida, para nossos propósitos atuais, pela:

1. A capacidade de o código ser hospedado em CI/CD:
 - a. Isso implica uma completa reestruturação dos códigos para que eles se tornem uma aplicação compartimentalizada. Será necessária uma total migração do paradigma do notebook para um paradigma mais robusto (como o Orientado ao Objeto).
2. A criação de um dashboard de gerenciamento geral de desempenho.

Portanto, vale ressaltar: os notebooks mostrados representam a estrutura atual de desenvolvimento e validação; a arquitetura de produção deverá posteriormente ser modularizada em código Python.

Uma diferença fundamental

Quando ambos os modelos, o modelo TFT preditivo e o modelo RL de gestão de banca, estiverem finalmente hospedados e ativos, diferentemente do que ocorre na arquitetura atual, teremos o seguinte workflow:

1. Ao receber os dados ao vivo do mercado e encontrar um padrão preditivo seguro, o modelo TFT gera um sinal.
 - a. Ela encontra uma “operação candidata”.
 - b. Depois, envia um sinal descrevendo a operação (altista/baixista, amplitude do movimento, probabilidade de concretização da previsão, etc.)
2. Este sinal é enviado ao envelope de risco.
 - a. O trabalho do envelope não é decidir se a previsão está correta.

- b. Ele usa uma janela temporal móvel (últimos 100 períodos, 200 períodos, 500 períodos, etc.) para abstrair os dados de distribuição de risco associado a previsão.
 - c. Como cada ferramenta gera uma array de dados (são 25 no total, distribuídos entre as 5 ferramentas), o resultado da análise da janela temporal gera uma concatenação de arrays de dados de risco.
3. Esta array é concatenada aos dados da “operação candidata”.
4. Depois disso, o modelo RL de gestão de banca recebe a concatenação e abre uma ordem com:
 - a. Direção.
 - b. Position size.
 - c. Stop loss.
 - d. Take profit.

Diretório básico e corrente do projeto Libellula

```
—/data
├──/artifacts
├──/raw
│   └── EUR_USD.csv # Exemplo de "Dataset original"
├──/processed # Datasets já trabalhados
│   ├──/combined # Datasets concatenados de risco
│   │   ├── audit_risk_features.ipynb
│   │   └── risk_fatures.parquet
│   ├──/cvar # Pasta do dataset de risco do CVaR
│   │   └── cvar_features.parquet
│   ├──/epf # Pasta do dataset de risco do EPF
│   │   └── epf_features.parquet
│   ├──/evt # Pasta do dataset de risco do EVT
│   │   └── evt_features.parquet
│   ├──/pdfd # Pasta do dataset de risco do PDFD
│   │   └── pdfd_features.parquet
│   ├──/qf # Pasta do dataset de risco do Quantile Forecasting
│   │   └── qf_features.parquet
│   └──/predictions # No momento está vazia (???)
└──/img
    └── libellula.jpeg
notebooks/
├── Predictive_TFT_model/
│   └── TFT_predictive_model.ipynb
├── financial_tools/
│   └── financial_tools.ipynb
├── risk_envelope/
│   ├── EPF.ipynb
│   ├── Quantile_Forecasting.ipynb
│   ├── CVaR.ipynb
│   ├── PDFD.ipynb
│   └── EVT.ipynb
```

```

├── concatenation/
│   └── Concatenation.ipynb
├── reinforcement_learning/ # ESTE ARQUIVO AINDA NÃO EXISTE!
│   └── RL.ipynb # ESTE NOTEBOOK AINDA NÃO EXISTE!
├── src
│   └── setup.py
├── .gitignore
└── README

```

Pastas e arquivos-chave do diretório

- `/data/`: nesta pasta ficam os datasets.
- `/data/raw/`: nesta pasta ficam os datasets ainda não “trabalhados” por nenhum processo do workflow, como, por exemplo, o *dataset original*.
- `/data/raw/EUR_USD.csv`: nesta pasta, a exemplo dos arquivos ‘EUR_USD.csv’, ficarão os *datasets originais*, que são compostos de cotações no formato OHLC básico e uma coluna índice de timestamp. Este é o “insumo” inicial do workflow do Libellula: cotações.
- `/data/processed/`: nesta pasta ficam os datasets originais que já passaram por algum processo algorítmico, como: implementação de indicadores técnicos, análise pelas ferramentas do envelope de risco ou pelo modelo TFT preditivo. Os datasets resultantes destes processos ficam nesta pasta.
- `/data/processed/combined/`: nesta pasta ficam os datasets gerados pelas ferramentas do envelope de risco já concatenadas pelo algoritmo de concatenação (‘/notebooks/risk_envelope/concatenation/Concatenation.ipynb’)
- `/data/processed/cvar/`: nesta pasta ficam os datasets gerados pelo algoritmo que calcula o CVaR das cotações iniciais.
- `/data/processed/epf/`: nesta pasta ficam os datasets gerados pelo algoritmo que calcula o EPF das cotações iniciais.
- `/data/processed/evt/`: nesta pasta ficam os datasets gerados pelo algoritmo que calcula o EVT das cotações iniciais.
- `/data/processed/pdfd/`: nesta pasta ficam os datasets gerados pelo algoritmo que calcula o PDFD das cotações iniciais.
- `/data/processed/qf/`: nesta pasta ficam os datasets gerados pelo algoritmo que calcula o Quantile Forecasting das cotações iniciais.
- `/data/processed/predictions/`: nesta pasta ficam os datasets gerados pelo modelo TFT preditivo (responsável por gerar previsões).
- `/notebooks/`: nesta pasta ficam os notebooks essenciais do projeto.
- `/notebooks/Predictive TFT model/`: nesta pasta ficam o notebook contendo o modelo TFT preditivo, responsável por analisar o dataset secundário (cotações em OHLC + indicadores técnicos). Este mesmo notebook contém um algoritmo que concatena dois datasets: o dataset gerado dentro do mesmo notebook (contendo as previsões do modelo preditivo) e o dataset gerado pelo envelope de risco (já

previamente concatenado) contendo as ferramentas do envelope de risco (gerado pelo algoritmo '/notebooks/concatenation/Concatenation.ipynb')

- `notebooks/financial_tools/`: nesta pasta ficam os notebook que cria os indicadores técnicos.
- `notebooks/risk_envelope/`: nesta pasta ficam os notebooks com as ferramentas do envelope de risco, responsável por receber as cotações do dataset inicial (localizado em '/data/raw/') e abstrair os dados probabilísticos de risco.
- `notebooks/risk_envelope/EPF.ipynb`: este é o arquivo que contém o notebook do algoritmo que recebe o dataset original e calcula o EPF.
- `notebooks/risk_envelope/Quantile_Forecasting.ipynb`: este é o arquivo que contém notebook do algoritmo que recebe o dataset original e calcula o Quantile Forecasting.
- `notebooks/risk_envelope/CVaR.ipynb`: este é o arquivo que contém o notebook do algoritmo que recebe o dataset original e calcula o CVaR.
- `notebooks/risk_envelope/PDFD.ipynb`: este é o arquivo que contém o notebook do algoritmo do algoritmo que recebe o dataset original e calcula o PDFD.
- `notebooks/risk_envelope/EVT.ipynb`: este é o arquivo que contém o notebook do algoritmo que calcula o EVT.
- `notebooks/concatenation/Concatenation.ipynb`: notebook contendo o algoritmo que concatena todos os datasets gerados pelas ferramentas do envelope de risco em um só dataset.

Alguns códigos fundamentais:

Método 'setup()'

Ele corrige eventuais erros de indexação, timestamps e labelling que porventura existam no *dataset original*:

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
import joblib
from pathlib import Path

def setup(
    csv_path,
    target_col="Price",
    horizon=1,
    train_ratio=0.7,
    val_ratio=0.15,
    save_artifacts=True,
    artifacts_path="../../../data/artifacts/",
    processed_path="../../../data/processed/"
):
    df = pd.read_csv(csv_path)
    df["Date"] = pd.to_datetime(df["Date"], format="%m/%d/%Y")
```

```

if "Change %" in df.columns:
    df["Change %"] = (
        df["Change %"]
        .str.replace("%", "", regex=False)
        .astype(float) / 100
    )

```

```

df = df.sort_values("Date").reset_index(drop=True)

```

```

df["target_return"] =
df[target_col].pct_change(horizon).shift(-horizon)
df["target_direction"] = (df["target_return"] > 0).astype(int)

```

```

df = df.dropna().reset_index(drop=True)

```

```

feature_cols = [col for col in df.columns if col not in [
    "Date",
    "target_return",
    "target_direction"
]]

```

```

X = df[feature_cols]
y = df["target_return"]

```

```

n = len(df)
train_end = int(n * train_ratio)
val_end = int(n * (train_ratio + val_ratio))

```

```

X_train, X_val, X_test = (
    X.iloc[:train_end],
    X.iloc[train_end:val_end],
    X.iloc[val_end:]
)

```

```

y_train, y_val, y_test = (
    y.iloc[:train_end],
    y.iloc[train_end:val_end],
    y.iloc[val_end:]
)

```

```

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_val_scaled = scaler.transform(X_val)
X_test_scaled = scaler.transform(X_test)

```

```

if save_artifacts:
    artifacts_dir = Path(artifacts_path)
    artifacts_dir.mkdir(parents=True, exist_ok=True)
    joblib.dump(scaler, artifacts_dir / "feature_scaler.pkl")

```

```

processed_dir = Path(processed_path)

```

```
processed_dir.mkdir(parents=True, exist_ok=True)
df.to_csv(processed_dir / "dataset_processed.csv",
index=False)
```

```
return {
    "X_train": X_train_scaled,
    "X_val": X_val_scaled,
    "X_test": X_test_scaled,
    "y_train": y_train.values,
    "y_val": y_val.values,
    "y_test": y_test.values,
    "feature_columns": feature_cols
}
```

Códigos do envelope de risco

Todos os códigos do envelope de risco seguem exatamente o mesmo roteiro.

1. Importação do dataset e bibliotecas
2. Execução do `setup()` e alinhamento temporal
3. Construção da ferramenta de risco
4. Sanity checks mínimos
5. DataFrame final (features de risco)
6. Padronização
7. Salvamento

Já que todos os códigos das ferramentas do envelope de risco seguem rigorosamente o mesmo protocolo, aqui será mostrado o código de apenas duas das ferramentas.

CVaR:

1 – Importação do dataset e bibliotecas:

```
import sys
from pathlib import Path
```

```
import numpy as np
import pandas as pd
```

```
sys.path.append(str(Path(".././src").resolve()))
from setup import setup
```

```
CSV_PATH =
"C:/projects/Libellula/data/processed/financial_tools_datset.csv"
TARGET_COL = "Price"
HORIZON = 1
TRAIN_RATIO = 0.7
VAL_RATIO = 0.15
```

2 – Execução do `setup()` e alinhamento temporal

```
raw_df = pd.read_csv(CSV_PATH)
```

```
raw_df["Date"] = pd.to_datetime(raw_df["Date"], format="%m/%d/%Y")
raw_df = raw_df.sort_values("Date").set_index("Date")
```

```
splits = setup(
    csv_path=CSV_PATH,
    target_col=TARGET_COL,
    horizon=HORIZON,
    train_ratio=TRAIN_RATIO,
    val_ratio=VAL_RATIO,
    save_artifacts=False,
)
```

```
processed_df = raw_df.reset_index().copy()
if "Change %" in processed_df.columns:
    processed_df["Change %"] = (
        processed_df["Change %"].str.replace("%", "",
        regex=False).astype(float) / 100
    )
```

```
processed_df = processed_df.sort_values("Date")
processed_df["target_return"] = (
    processed_df[TARGET_COL].pct_change(HORIZON).shift(-HORIZON)
)
processed_df = processed_df.dropna().set_index("Date")
```

```
n_rows = len(processed_df)
train_end = int(n_rows * TRAIN_RATIO)
val_end = int(n_rows * (TRAIN_RATIO + VAL_RATIO))
val_index = processed_df.iloc[train_end:val_end].index
```

```
# print("Validation length:", len(val_index))
```

```
print("Train shape:", splits["X_train"].shape)
print("Val shape:", splits["X_val"].shape)
print("Test shape:", splits["X_test"].shape)
```

3 – Construção da ferramenta de risco

```
returns = processed_df[TARGET_COL].pct_change().fillna(0.0)
```

```
def cvar_from_window(window_values: np.ndarray, alpha: float) ->
float:
    q = np.quantile(window_values, alpha)
    tail = window_values[window_values <= q]
    if tail.size == 0:
        return float(q)
    return float(tail.mean())
```

```
rolling_window = 100
cvar_95_full = returns.rolling(rolling_window,
min_periods=20).apply(
    lambda x: cvar_from_window(x.values, alpha=0.05),
    raw=False,
```

```
)
cvar_99_full = returns.rolling(rolling_window,
min_periods=20).apply(
    lambda x: cvar_from_window(x.values, alpha=0.01),
    raw=False,
)

tail_loss_expectation_full = (0.5 * cvar_95_full) + (0.5 *
cvar_99_full)

cvar_dataset = pd.DataFrame(index=val_index)
cvar_dataset["cvar_95"] = cvar_95_full.reindex(val_index)
cvar_dataset["cvar_99"] = cvar_99_full.reindex(val_index)
cvar_dataset["tail_loss_expectation"] =
tail_loss_expectation_full.reindex(val_index)

cvar_dataset = cvar_dataset.bfill().sort_index()
```

4 – Sanity checks mínimos

```
print("NaN críticos:")
print(cvar_dataset.isna().sum())

print("\nDistribuição plausível:")
print(cvar_dataset.describe().T[["mean", "std", "min", "max"]])

print("\nAlinhamento temporal:")
print("Index monotonic increasing:",
cvar_dataset.index.is_monotonic_increasing)
print("Index has duplicates:", cvar_dataset.index.has_duplicates)

print("Dataset lines == val_index lines:", len(cvar_dataset) ==
len(val_index))
print("Dataset index == val_index:",
cvar_dataset.index.equals(val_index))
```

5 – DataFrame final (features de risco)

```
cvar_dataset = cvar_dataset[["cvar_95", "cvar_99",
"tail_loss_expectation"]]
cvar_dataset.tail(), cvar_dataset.shape

cvar_dataset.index
```

6 – Padronização e salvamento

```
def standardize_dataset(df):
    df = df.sort_index().bfill()

    df.index = pd.to_datetime(df.index)
    df.index = df.index.astype("datetime64[us]")
    df.index.name = "timestamp"
```

```
df = df.astype(np.float32)
```

```
assert isinstance(df.index, pd.DatetimeIndex)
assert df.index.dtype == "datetime64[us]"
assert df.index.name == "timestamp"
assert df.index.is_monotonic_increasing
assert not df.index.has_duplicates
```

```
return df
```

7 - Salvamento

```
OUTPUT_PATH =
```

```
"C:/projects/Libellula/data/processed/cvar/cvar_features.parquet"
```

```
cvar_dataset.to_parquet(OUTPUT_PATH, index=True)
```

```
OUTPUT_PATH
```

EPF:

1 - Importação do dataset e bibliotecas

```
import sys
from pathlib import Path
```

```
import numpy as np
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score
```

```
sys.path.append(str(Path("../..").resolve())) # Correção do
path: adicionada mais uma camada de pasta.
from setup import setup
```

```
CSV_PATH =
"C:/projects/Libellula/data/processed/financial_tools_datset.csv"
# IDEM!!!
TARGET_COL = "Price"
HORIZON = 1
```

2 - Execução do `setup()` e alinhamento temporal

```
raw_df = pd.read_csv(CSV_PATH)
raw_df["Date"] = pd.to_datetime(raw_df["Date"], format="%m/%d/%Y")
raw_df = raw_df.sort_values("Date").set_index("Date")
```

```
splits = setup(
    csv_path=CSV_PATH,
    target_col=TARGET_COL,
    horizon=HORIZON,
)
```

```
n_rows = len(raw_df)
train_end = len(splits["X_train"])
val_end = train_end + len(splits["X_val"])
```

```
val_index = raw_df.iloc[train_end:val_end].index
```

```
print("Train shape:", splits["X_train"].shape)
print("Val shape:", splits["X_val"].shape)
print("Test shape:", splits["X_test"].shape)
```

3 – Construção da ferramenta de risco

```
y_train_bin = (splits["y_train"] > 0).astype(int)
y_val_bin = (splits["y_val"] > 0).astype(int)
```

```
model = LogisticRegression(max_iter=1000)
model.fit(splits["X_train"], y_train_bin)
```

```
val_probs = model.predict_proba(splits["X_val"])[ :, 1]
auc = roc_auc_score(y_val_bin, val_probs)
print("Validation AUC:", auc)
```

```
epf_probs = pd.Series(val_probs, index=val_index, name="epf_prob")
epf_score = epf_probs.rolling(50).mean()
```

```
price = raw_df.loc[val_index, TARGET_COL]
epf_upper = price * (1 + epf_score)
epf_lower = price * (1 - epf_score)
```

```
epf_dataset = pd.DataFrame(index=val_index)
epf_dataset["epf_upper"] = epf_upper
epf_dataset["epf_lower"] = epf_lower
epf_dataset["epf_width"] = epf_dataset["epf_upper"] -
epf_dataset["epf_lower"]
mid = (epf_dataset["epf_upper"] + epf_dataset["epf_lower"]) / 2
epf_dataset["epf_asymmetry"] = (price - mid) /
epf_dataset["epf_width"]
```

4 – Sanity checks mínimos

```
print("NaN críticos:")
print(epf_dataset.isna().sum())
```

```
print("\nAlinhamento temporal:")
print("Index monotonic increasing:",
epf_dataset.index.is_monotonic_increasing)
print("Index has duplicates:", epf_dataset.index.has_duplicates)
print("Dataset lines == val_index lines:", len(epf_dataset) ==
len(val_index))
```

5 – DataFrame final (features de risco)

```
epf_dataset = epf_dataset[["epf_upper", "epf_lower", "epf_width",
"epf_asymmetry"]]
epf_dataset.head(), epf_dataset.shape
```

6 – Padronização

```
def standardize_dataset(df):
```

```
df = df.sort_index().bfill()
```

```
df.index = pd.to_datetime(df.index)
df.index = df.index.astype("datetime64[us]")
df.index.name = "timestamp"
```

```
df = df.astype(np.float32)
```

```
assert isinstance(df.index, pd.DatetimeIndex)
assert df.index.dtype == "datetime64[us]"
assert df.index.name == "timestamp"
assert df.index.is_monotonic_increasing
assert not df.index.has_duplicates
```

```
return df
```

```
epf_dataset = standardize_dataset(epf_dataset)
```

7 – Salvamento

```
OUTPUT_PATH =
"C:/projects/Libellula/data/processed/epf/epf_features.parquet"
```

```
epf_dataset.to_parquet(OUTPUT_PATH, index=True)
```

```
OUTPUT_PATH
```

Modelo TFT preditivo

1 – Importações

```
import pandas as pd
import numpy as np
import torch
```

```
from pytorch_forecasting import TimeSeriesDataSet,
TemporalFusionTransformer
from pytorch_forecasting.metrics import QuantileLoss
```

```
import lightning.pytorch as pl
from pytorch_lightning.callbacks import EarlyStopping,
LearningRateMonitor
```

```
import matplotlib.pyplot as plt
```

2 – Loading de dataset original

```
# Carregar dataset base (OHLC + indicadores)
df =
pd.read_csv("C:/projects/Libellula/data/processed/financial_tools_
dataset.csv")
```

```
# Ordenar temporalmente (CRÍTICO)
df = df.iloc[::-1].reset_index(drop=True)
```

```

# Converter data
df["Date"] = pd.to_datetime(df["Date"])

# Criar índice temporal numérico (exigência do TFT)
df["time_idx"] = np.arange(len(df))

# Criar target → retorno futuro (melhor que preço)
df["target"] = df["Price"].pct_change().shift(-1)

# Remover NaNs
df = df.dropna().reset_index(drop=True)

# ID da série (necessário para TFT)
df["series"] = "asset_1"

df.head()

```

3 – Configuração temporal

```

max_encoder_length = 64
max_prediction_length = 1

```

4 – Criação do dataset do TFT

```

cols_obj = df.select_dtypes(include=["object"]).columns

for col in cols_obj:
    df[col] = (
        df[col]
        .astype(str)
        .str.replace("%", "", regex=False)
        .str.replace(",", ".", regex=False)
        .str.extract(r"([+-]?\d*\.\d+)"[0] # <-- pega só número
    )
    df[col] = pd.to_numeric(df[col], errors="coerce")

df = df.dropna().reset_index(drop=True)

df.select_dtypes(include=["object"])

# Definir colunas que o modelo usa
training = TimeSeriesDataSet(
    df,
    time_idx="time_idx",
    target="target",
    group_ids=["series"],

    max_encoder_length=max_encoder_length,
    max_prediction_length=max_prediction_length,

    time_varying_known_reals=["time_idx"],

    time_varying_unknown_reals=[

```

```

        col for col in df.columns
        if col not in ["Date", "series", "target"]
    ]
)

```

5 – Dataloader

```

train_dataloader = training.to_dataloader(
    train=True,
    batch_size=64,
    num_workers=0
)

```

6 – Criar modelo TFT

```

tft = TemporalFusionTransformer.from_dataset(
    training,
    learning_rate=0.001,
    hidden_size=32,
    attention_head_size=4,
    dropout=0.1,
    hidden_continuous_size=16,
    loss=QuantileLoss(),
    log_interval=10,
    reduce_on_plateau_patience=4,
)

```

7 – Callbacks

```

early_stop = EarlyStopping(
    monitor="train_loss",
    patience=5,
    mode="min"
)

```

```

lr_logger = LearningRateMonitor()

```

8 – Trainer

```

trainer = pl.Trainer(
    max_epochs=30,
    gradient_clip_val=0.1,
    callbacks=[early_stop, lr_logger]
)

```

9 – Treinamento

```

trainer.fit(
    tft,
    train_dataloaders=train_dataloader
)

```

10 – Gerar previsões

```

raw_predictions = tft.predict(
    train_dataloader,
    mode="raw"
)

```

```
pred = raw_predictions["prediction"].numpy()
mean_pred = pred.mean(axis=2)
```

11 – Criar dataset de previsões

```
# Criar índice alinhado com saída do modelo
pred_index = df.index[-len(mean_pred):]
```

```
df_pred = pd.DataFrame(index=pred_index)
```

```
df_pred["prediction"] = mean_pred.flatten()
```

```
# retorno previsto (já é target, mas mantemos consistência)
df_pred["pred_return"] = df_pred["prediction"]
```

```
# confiança simples (magnitude)
df_pred["confidence"] = np.abs(df_pred["pred_return"])
```

```
df_pred.head()
```

O que vem agora?

Agora, falta cumprir algumas etapas fundamentais para o projeto. Em ordem de importância, eu diria:

1. *Modelo RL de gestão de banca*: **nem começamos a programa-lo e ele é a peça conceitualmente mais fundamental de todo o projeto!** Precisamos cria-lo, definir os parâmetros de sucesso de um teste de desempenho, e aí testá-lo.
2. Isto feito, temos que reexecutar todos os códigos do projeto para identificar eventuais defeitos de arquitetura.
3. Depois de todos os códigos estarem prontos (dentro do paradigma estático do notebook), eles devem ser migrados para um paradigma mais robusto que permita a implementação em hospedagem em CI/CD.
4. Só então, iremos transformar nossos notebooks em software real, baseado em um ecossistema de tecnologias especializadas.

Workflow do Libellula

